DM-Call: Decision-Model-Guided Function Calling with Cumulative Verification Feedback

Minhyeong Yu*
Enhans
Seoul, South Korea
minhyeong@enhans.ai

Wonduk Seo*
University College London
London, United Kingdom
wonduk.seo.26@ucl.ac.uk

Seunghyun Lee✉
Enhans
Seoul, South Korea
seunghyun@enhans.ai

arXiv:2611.10977v1 [cs.IR] 7 Nov 2026

Abstract

Reliable function calling requires Large Language Models (LLMs) to select appropriate functions and generate arguments consistent with user intent. However, syntactically valid calls may still be semantically incorrect due to inappropriate function selection or mismatched arguments. In this paper, we propose **Decision-Model-Guided Function Calling (DM-Call)**, a framework that combines local schema validation with separate decision-model verification of function selection and argument correctness. For rejected candidates, **DM-Call** accumulates verification probabilities, binary judgments, and schema errors across attempts. A hybrid feedback representation further combines numerical probabilities with verbal correctness labels to guide iterative refinement without executing external tools. Extensive evaluation across BFCL and API-Bank (first-call) demonstrates **DM-Call** boosts function-calling accuracy versus direct generation, at modest additional cost.[1]

CCS Concepts

• **Information systems** → **Query intent**; **Information extraction**; **Recommender systems**; **Retrieval efficiency**; **Decision support systems**.

Keywords

function calling, decision model, evaluation, in-context learning

ACM Reference Format:
Minhyeong Yu, Wonduk Seo, and Seunghyun Lee. 2018. DM-Call: Decision-Model-Guided Function Calling with Cumulative Verification Feedback. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

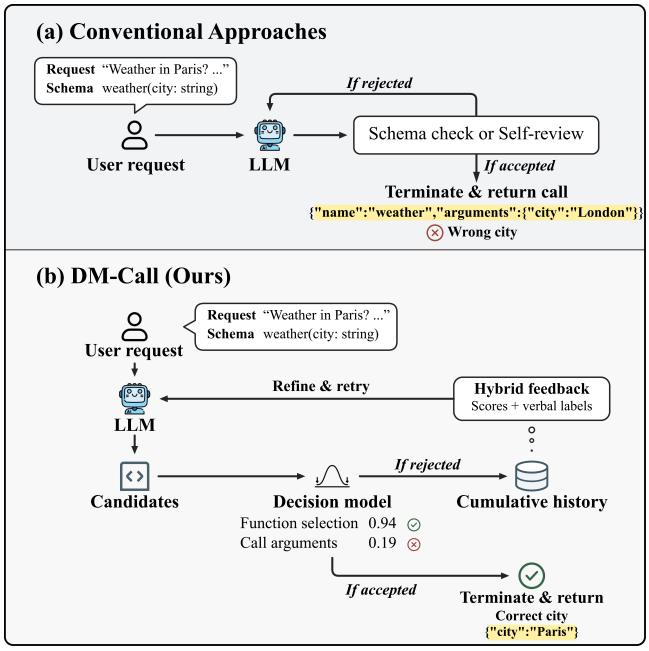


Figure 1: Comparison of conventional function-calling approaches and DM-Call. (a) Conventional approaches rely on schema validation or self-review, which may fail to detect semantically incorrect calls. (b) DM-Call separately verifies function selection and argument correctness through probabilistic judgments, accumulating rejected candidates and their hybrid feedback to guide subsequent revisions.

1 Introduction

Function calling enables Large Language Models (LLMs) to interact with external tools and APIs by translating natural-language requests into structured function invocations [3, 7]. Specifically, given a user request and available functions, an LLM determines whether a function call is needed and, if so, selects an appropriate function and generates arguments that reflect user intent. However, syntactically valid calls may still be semantically incorrect due to inappropriate function selection or mismatched arguments [6].

Prior work has explored various approaches to improving function-calling reliability through validation and iterative refinement. Self-Refine revises candidate outputs using model-generated feedback [3]. Other approaches, such as multi-agent debate and Reinforced Agent, incorporate feedback from multiple agents or a separate reviewer [2, 9]. More closely related, ToolVerifier applies self-verification to tool selection and parameter generation [5].

Despite this progress, several limitations remain: (1) syntactic constraints and schema validation alone cannot determine whether function selection and arguments align with user intent, (2) generic self-refinement and multi-agent approaches may incur additional

*Both authors contributed equally to this research.
[1]Code is available at: github.com/gogoymh/decision-model-guided-function-calling

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

inference overhead without explicitly distinguishing errors in function selection from those in argument generation, and (3) refinement strategies that rely solely on the latest feedback may overlook useful verification signals from earlier attempts.

To address these limitations, we propose **Decision-Model-Guided Function Calling (DM-Call)**, a framework that combines local schema validation with separate decision-model judgments of function selection and argument correctness, as illustrated in Figure 1. When a candidate fails verification, **DM-Call** retains the rejected call along with its verification probabilities, binary judgments, and schema errors. To guide subsequent revisions, we introduce a *hybrid feedback representation* that combines numerical verification scores with verbal correctness labels. By accumulating these signals across rejected attempts, **DM-Call** enables the generator to iteratively refine candidate calls using feedback from previous failures.

We evaluate **DM-Call** using four diverse LLMs and two decision-model verifiers on the Berkeley Function Calling Leaderboard (BFCL) single-turn cases [6] and an adapted API-Bank first-call benchmark [3]. **DM-Call** improves function-calling accuracy and inference costs over baselines. Moreover, ablation studies demonstrate the benefits of cumulative feedback while examining verification components and feedback representations, alongside accuracy–latency trade-offs across repair budgets.

Our core contributions include (1) a decision-model-guided function calling framework that combines schema validation with separate verification of function selection and argument correctness, (2) a cumulative feedback mechanism that preserves rejected calls and their verification signals for iterative refinement, and (3) a hybrid verification-feedback representation that combines numerical verification probabilities with verbal correctness labels to guide subsequent revisions.

2 Methodology

2.1 Task Definition

Given a user request x and the available functions $\mathcal{F}$ with their schemas, a system returns one next action: a call $c = (f, a)$ with function $f \in \mathcal{F}$ and arguments a, or no call, $c = \varnothing$. The request may include earlier dialogue turns. The action is correct when it matches the benchmark reference, that is, the expected function with acceptable argument values, or no call when none is expected.

It can be wrong in three specific ways: (1) it calls a function when no call is expected, or returns no call when one is; (2) it picks the wrong function; or (3) its arguments are wrong or missing. Because even a schema-valid wrong call can trigger an unintended external action, **DM-Call** verifies and refines candidate calls before execution, without tool-execution feedback.

2.2 Decision-Model Verification

DM-Call separates an LLM generator G, which proposes candidate calls, from a decision model D, which verifies them. At iteration t, the generator produces c_t conditioned on the request, function schemas, and a history H_t of rejected candidates and their feedback. The decision model D asks two yes/no questions about c_t and returns the probability of *yes* for each, $p_f^{(t)}, p_a^{(t)} \in [0, 1]$:

$$c_t = G(H_t, x, \mathcal{F}), \qquad p_f^{(t)} = D_f(x, \mathcal{F}, c_t), \quad p_a^{(t)} = D_a(x, \mathcal{F}, c_t). \tag{1}$$

The function question D_f assesses whether the selected function matches the request or, for a no-call candidate proposed during repair, whether no call is appropriate. The argument question D_a assesses whether the required values are present, supported by the request, and assigned to the correct parameters. For a no-call candidate, argument verification is skipped and $p_a^{(t)} = \perp$ (a null value). A parseable call with missing arguments still receives both judgments. As an exception, if the initial candidate c_0 is a parseable no-call, **DM-Call** returns it without verification or repair and marks it as unverified. An unparseable output is not treated as a no-call.

2.3 Acceptance and Cumulative Repair

DM-Call accepts a candidate only when it passes a schema check and every decision-model question that applies to it. The schema check $S(c_t)$ is true when c_t is readable, names a function in $\mathcal{F}$ and supplies every required argument with its declared type. A question that is not asked, such as the argument question on a no call or any question on an unreadable candidate, has the missing score $\perp$. The verdict $V(p)$ is true if and only if $p \neq \perp$ and $p \geq 0.5$. Candidate c_t is accepted, written $A_t = \text{true}$, when

$$A_t = \begin{cases} S(c_t) \wedge V\big(p_f^{(t)}\big), & c_t \text{ is a no call,} \\ S(c_t) \wedge V\big(p_f^{(t)}\big) \wedge V\big(p_a^{(t)}\big), & \text{otherwise,} \end{cases} \tag{2}$$

where $\wedge$ denotes logical AND. A readable candidate that fails S is still judged by D, so its feedback carries both the schema errors and the decision model's judgments.

When c_t is rejected (A_t is false) and a repair remains ($t < r$), **DM-Call** queries the generator for a revised candidate $c_{t+1} = G(H_{t+1}, x, \mathcal{F})$ (Eq. 1). The history H_t lists the candidates rejected before step t, oldest first, each paired with its feedback d_i:

$$\begin{aligned} H_t &= \big[(c_0, d_0), \ldots, (c_{t-1}, d_{t-1})\big], \\ d_i &= \big(\Phi(p_f^{(i)}),\ \Phi(p_a^{(i)}),\ V(p_f^{(i)}),\ V(p_a^{(i)}),\ S(c_i)\big). \end{aligned} \tag{3}$$

Here Φ denotes the feedback representation used to convey verification scores to the generator, while the remaining entries capture the binary judgments and schema validation results, including any error messages.[2] The generator receives these records, explicitly marked as unexecuted, alongside the original request. In contrast, the decision model evaluates only the current candidate against the request and function schemas. **DM-Call** returns the first accepted candidate or falls back to the initial candidate c_0 if none passes verification within the repair budget.

2.4 Hybrid Feedback

The feedback representation Φ in Eq. 3 determines how verification scores are conveyed to the generator. Our default representation, *Hybrid*, pairs each probability with a verbal label:

$$\begin{aligned} \Phi(p) &= \big(p, \mathcal{L}_{k(p)}\big), \quad \Phi(\perp) = (\perp, \perp), \\ k(p) &= \min\big(\lfloor 5p \rfloor + 1,\ 5\big). \end{aligned} \tag{4}$$

[2] Section 2.4 describes the default *hybrid* feedback representation.

Table 1: Comparison of function-calling methods across four LLM generators on BFCL and API-Bank. We report three-run mean accuracy (%) at $r^\star = 3$ (s.d. 0.00–2.89) and total inference cost (USD). Decision. Generation refers to function-call generation guided by separate decision-model verification. Bold and underline indicate the best and second-best results per generator and metric, respectively; – denotes unpriced local inference. †API-Bank uses first-call exact match without no-call negatives.

	gpt-5-nano			gpt-5.4-mini			claude-sonnet-5.5			gemma-4-31b-it		
Method	BFCL↑	API-Bank† ↑	USD↓	BFCL↑	API-Bank† ↑	USD↓	BFCL↑	API-Bank† ↑	USD↓	BFCL↑	API-Bank† ↑	USD↓
Direct Generation												
Direct	64.12	61.32	**0.167**	81.59	80.13	**2.146**	88.25	86.97	**12.598**	86.67	81.41	–
Self-Refine	69.45	58.55	0.762	81.27	79.91	9.591	88.43	**87.39**	56.184	86.69	81.41	–
Multi-Agent												
MAD-FC	63.53	66.03	1.493	81.10	**80.98**	16.501	85.22	81.62	95.244	84.91	**82.48**	–
PF-FC	74.54	**70.09**	5.903	81.79	76.71	6.750	86.45	83.12	18.686	84.85	77.56	–
Decision. Generation												
DM-Call (*Jev*)	**74.63**	64.32	0.525	**82.54**	80.13	2.783	**88.56**	**87.39**	13.988	**86.91**	81.62	–
DM-Call (*PPLX*)	74.24	64.53	–	82.20	80.13	–	88.46	**87.39**	–	86.88	81.41	–

Here, $\mathcal{L} = (L_1, \ldots, L_5)$ denotes the ordered labels *very likely incorrect*, *likely incorrect*, *uncertain*, *likely correct*, and *very likely correct*. Each label corresponds to a probability bin of width 0.2, ordered by increasing p, with $p = 1$ assigned to the final bin. The representation affects only the feedback provided to the generator, while verification decisions continue to use the numeric probabilities through V (Eq. 2).[3]

3 Experiments

3.1 Experimental Setup

Datasets and Metrics. For BFCL, we use 3,035 single-turn Python cases, including irrelevance cases that expect no call, scored by the official AST metric.[4] For API-Bank [3], we use 156 first-call test cases with a fixed 53-API catalog, scored by exact match of function names and type-normalized arguments without executing APIs.

Models. We evaluate four LLM generators: (i) `gpt-5-nano`, (ii) `gpt-5.4-mini` [8], (iii) `claude-sonnet-5.5`[5], and (iv) the 31B instruction tuned variant of `gemma-4` [10]. Reasoning effort is set to `none` where supported, with no additional thinking mode enabled. For verification, we employ two decision models: the hosted *Jev* (`jev-1.13.0`)[6] and the open-weight 27B *PPLX Decider v1.1* [1].

Baselines and DM-call (Ours). We consider two baseline categories (Table 1). *Direct Generation* includes (i) *Direct*, which returns the initial prediction, and (ii) *Self-Refine* [4], which iteratively revises predictions using self-generated critiques for up to three rounds. *Multi-Agent* includes (iii) *MAD-FC* [2], which uses three agents for one debate round with majority voting, and (iv) *PF-FC* [9], which employs an LLM reviewer for up to three refinement rounds.[7]

For **DM-Call**, we employ separate function and argument verification with a threshold of 0.5 and up to three repair rounds ($r^\star = 3$).

[3]Section 3.3 compares *Hybrid* with *Numeric* and *Text* feedback.
[4]We exclude 16 live-relevance cases that test only whether a call is made.
[5]https://www.anthropic.com/claude-sonnet-5-5
[6]https://typesafe.ai/blog/introducing-system-one-models-and-jev
[7]All methods except MAD-FC share the initial Direct prediction; MAD-FC also recognizes tool calls expressed as text.

Table 2: Ablation study of DM-Call on BFCL using `gpt-5-nano` and *Jev* ($r = 3$), reporting three-run mean accuracy (%) ± standard deviation for overall function calling, function selection, and argument correctness. Bold and underline indicate the best and second-best results per metric, respectively.

Method	Call (%)	Func. (%)	Args. (%)
Direct	64.11 ± 0.31	75.11 ± 0.29	81.27 ± 0.14
(i) DM-Call-Function	73.12 ± 0.46	83.84 ± 0.52	81.78 ± 0.07
(iii) DM-Call-Argument	73.61 ± 0.33	83.73 ± 0.52	**82.64** ± 0.38
(iii) DM-Call-Latest	73.05 ± 0.24	83.61 ± 0.22	82.05 ± 0.16
DM-Call (Default)	**74.32** ± 0.22	**84.80** ± 0.26	82.18 ± 0.13

Table 3: Comparison of Numeric, Text, and Hybrid feedback representations. Results show three-run mean accuracy (%) (s.d. 0.02–0.54), with overall-averages across generators. Bold and underline indicate the highest and second-highest average scores per decision model, respectively.

	Jev			*PPLX*		
Generator	Numeric	Text	Hybrid	Numeric	Text	Hybrid
`gpt-5-nano`	74.20	74.14	74.63	73.95	73.68	74.24
`gpt-5.4-mini`	82.45	82.47	82.54	82.22	82.26	82.20
`claude-sonnet-5.5`	88.53	88.57	88.56	88.45	88.46	88.46
`gemma-4-31b-it`	86.93	86.91	86.91	86.89	86.91	86.88
Avg. Score	83.03	83.02	**83.16**	82.88	82.83	**82.95**

3.2 Main Results

DM-Call achieves the highest BFCL accuracy across all four generators (Table 1), with particularly substantial gains for `gpt-5-nano` (+10.5 percentage points over Direct). In contrast, conventional refinement and multi-agent baselines show less consistent improvements, often at higher inference costs. On API-Bank, performance gains are more limited, with **DM-Call** achieving competitive results while several baselines perform better on individual generators. Overall, **DM-Call** offers a favorable accuracy–cost trade-off, particularly for weaker generators, by selectively refining candidates that fail verification.

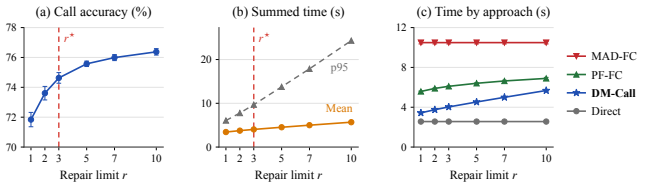


Figure 2: Effect of repair budget r on DM-Call using `gpt-5-nano`, BFCL, and *Jev*. (a) Call accuracy, (b) mean and p95 latency, and (c) latency comparison with baselines.

3.3 Ablation Study

Verification and history. Table 2 examines the contributions of separate verification and cumulative feedback. **(i) DM-Call-Function** and **(ii) DM-Call-Argument** evaluate only function selection or argument correctness, respectively, while **(iii) DM-Call-Latest** retains only the most recent rejection. All variants outperform Direct by 8.9–10.2 percentage points. The full **DM-Call** achieves the highest overall call accuracy with low standard deviations across runs, whereas **(ii) DM-Call-Argument** yields the best argument accuracy. Using only the latest feedback reduces call accuracy by 1.3 points, highlighting the benefit of retaining earlier verification feedback.

Feedback representation. Table 3 compares three feedback representations: *Numeric*, which provides verification probabilities; *Text*, which uses the verbal labels in Eq. 4; and *Hybrid*, which combines both. Hybrid yields the largest gains for `gpt-5-nano`, where repairs are more frequent (24% with *Jev* and 21% with *PPLX*), outperforming Numeric by 0.3–0.4 points and Text by 0.5–0.6 points. For the remaining generators, which require fewer repairs (1–6%), differences among the representations remain within 0.1 points. Overall, Hybrid achieves the highest average accuracy with both decision models, supporting its use as the default representation.

Repair budget. Figure 2 shows the accuracy–latency trade-off across repair budgets. Increasing r from 3 to 10 improves accuracy by only 1.8 points, while p95 latency rises from 9.7 to 24.3 s. At $r = 3$, **DM-Call** achieves a mean latency of 4.0 s, compared with 2.6 s for Direct, 6.1 s for PF-FC, and 10.5 s for MAD-FC. We select $r^\star$ using an elbow-style criterion, $r^\star = \arg\max_{r \in R}(\widetilde{\mathrm{Acc}}_r - \widetilde{C}_r)$, where $R = \{1, 2, 3, 5, 7, 10\}$ and $\widetilde{X}_r = (X_r - X_1)/(X_{10} - X_1)$ normalizes accuracy and requests per case. This yields $r^\star = 3$; increasing to $r = 5$ adds only 0.9 points but requires 16% more requests and 42% higher p95 latency.

4 Case Study

Table 4 presents four representative repair trajectories of **DM-Call** with *Jev*. The cases illustrate how verification guides refinement. In (a), a low p_f redirects Sonnet to the correction, while in (b), a low p_a prompts `gpt-5.4-mini` to correct the rating threshold. In (c), `gpt-5-nano` switches to a no-call response, whereas (d) demonstrates multi-step refinement leading to the correct call.

Table 4: Case studies of DM-Call with *Jev* at $r^\star = 3$, illustrating function correction, argument refinement, no-call decisions, and multi-step repair. Each step reports prob. (p_f, p_a) and correctness, with – indicating skipped argument verification.

t	Proposed call c_t	p_f	p_a	Correct
(a) Function. `claude-sonnet-5.5`, BFCL: "how to cook sikhae"				
0	`HNA_WQA_search(keyword="how to make sikhae Korean sweet rice drink recipe")`	0.37	0.94	✗
1	`cookbook_search_recipe(keyword="sikhae")`	0.93	0.95	✓
(b) Argument value. `gpt-5.4-mini`, BFCL: "Find me the best Italian restaurants in New York City with average customer ratings of more than 4 and accepts credit cards."				
0	`restaurant_search(accepts_credit_cards=true, cuisine="Italian", location="New York City, NY", rating=5)`	0.94	0.19	✗
1	`restaurant_search(. . . , rating=4)`	0.96	0.57	✓
(c) No call. `gpt-5-nano`, BFCL: a question in Chinese, "Who is Musk?", with only a tax calculator				
0	`calculate_tax(filing_status="single", income=0)`	0.03	0.12	✗
1	no call; answers in text	0.97	–	✓
(d) Several repairs. `gpt-5-nano`, API-Bank: "Can you help me delete my account?" … "My username is 'foo', password is 'bar'." Assistant: "Alright, let me check if the token you provided is valid."				
0	`CheckToken(token="")`	0.28	0.07	✗
1	`CheckToken(token="")`	0.33	0.06	✗
2	`DeleteAccount(token="")`	0.43	0.06	✗
3	`GetUserToken(username="foo", password="bar")`	0.84	0.96	✓

5 Conclusion

DM-Call combines separate decision-model verification of function selection and argument correctness with iterative refinement using cumulative Hybrid feedback, improving both function-calling accuracy and cost-effectiveness.

References

[1] Bespoke Labs and Maheswaran Sathiamoorthy. 2026. Nimble. https://github.com/bespokelabsai/nimble. Model: https://huggingface.co/bespokelabs/Bespoke-Nimble-9B.

[2] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. *arXiv preprint arXiv:2305.14325* (2023).

[3] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 3102–3116.

[4] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems* 36 (2023), 46534–46594.

[5] Dheeraj Mekala, Jason E Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. 2024. Toolverifier: Generalization to new tools via self-verification. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 5026–5041.

[6] Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

[7] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.

[8] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267* (2025).

[9] Anh Ta, Junjie Zhu, and Shahin Shayandeh. 2026. Reinforced Agent: Inference-Time Feedback for Tool-Calling Agents. In *Proceedings of the Fifth Workshop on Generation, Evaluation and Metrics (GEM)*. 136–147.

[10] Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, et al. 2026. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770* (2026).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009